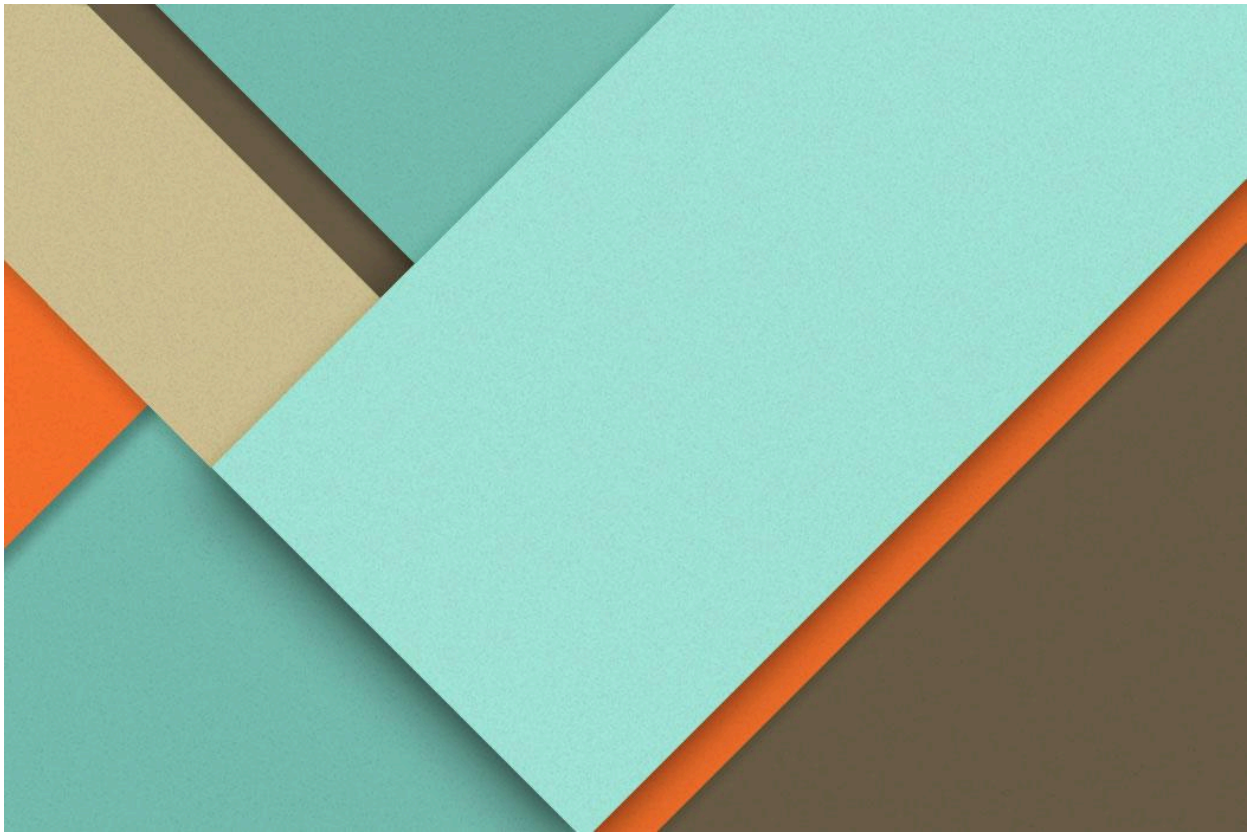

Smart Router Enterprise Readiness: Technical & Enterprise Assessment

CTO assessment for Magma

—

Lava RPC Smart Router — evaluation and hardening for custodians, exchanges, and wallet infrastructure providers

Guy Harry

System Architecture

What the Smart Router Is

The RPC Smart Router is Lava's centralized RPC gateway for enterprises with known provider infrastructure.
Unlike the decentralized rpcconsumer (on-chain pairing, stake-weighted selection), the Smart Router is config-driven: static and backup providers are defined in YAML, and there is no blockchain dependency for provider discovery or epochs.
This makes it suitable for custodians, exchanges, and wallet providers who control or trust specific RPC endpoints (e.g. Alchemy, Infura, self-hosted nodes) and want predictable routing, backup failover, and QoS-based selection without chain overhead.

High-Level Data Flow

User/dApp Request

↓

Smart Router (lavap rpcsmartrouter) — listens on configured network addresses per chain/API

↓

ChainListener (JSON-RPC | REST | TendermintRPC | gRPC) — parses request, builds protocol message

↓

SendRelay / SendParsedRelay → ProcessRelaySend

↓

SmartRouterRelayStateMachine — selection mode (Stateless / Stateful / CrossValidation), retry policy

↓

relaycore.RelayProcessor — task channel, result aggregation, consistency

↓

Provider selection (ConsumerSessionManager + ProviderOptimizer: balanced, latency, sync-freshness, etc.)

↓

Static providers (primary) → on failure → Backup providers

↓

Optional: **cache layer**, then **response back** to user

Main Components

Component	Role
RPCSmartRouter	Top-level process: loads config, creates per-endpoint servers, session managers, epoch timer, optimizers, consistency; starts chain listeners.
RPCSmartRouterServer	Per-endpoint server: implements `SendRelay`/`ParseRelay`/`SendParsedRelay`, uses `ChainListener`, handles relay flow, cache, consistency, subscriptions.
SmartRouterRelayStateMachine	Per-request state machine: selection mode (Stateless vs Stateful vs CrossValidation), retry logic (shouldRetry/retryCondition), relay state transitions, archive upgrade, timeout handling.
relaycore	Shared with rpcconsumer: `RelayProcessor`, `RelayState`, `ResultsManager`, `Consistency`, selection and result aggregation.
lavasession	`ConsumerSessionManager`, `UsedProviders`; static providers are converted from config (e.g. `convertProvidersToSessions` in `rpcsmartrouter.go`).

Selection is determined per request:
Stateless = single provider with retries on failure;
Stateful = all top providers at once, no retries;
CrossValidation = multiple providers, agreement threshold, no retries.

This directly affects reliability and failure modes.

Contrast with RPC Consumer

Aspect	Smart Router	RPC Consumer

Provider discovery	Static + backup from config	On-chain pairing
Trust model	Trust known providers	Trustless (chain-verified)
Retry / failover	Config-driven static/backup + QoS	Stake-weighted + QoS
Blockchain dependency	Optional (specs only)	Required

Critical Technical Risks (3–5)

Retry and "Ban" Logic

In Stateless mode the state machine may retry up to 10 times (MaximumNumberOfTickerRelayRetries). Failed relays are then "banned" via RelayRetriesManager: a ristretto cache keyed by request hash with a 6-hour TTL. Once a request shape has exhausted retries, that hash is added to the cache and any identical request will not be retried for 6 hours.

Code (abbreviated):

```
// protocol/lavaprotocol/relay_retries_manager.go
const RetryEntryTTL = 6 * time.Hour
func (rrm *RelayRetriesManager) AddHashToCache(hash string) {
    rrm.cache.SetWithTTL(hash, struct{}{}, 1, RetryEntryTTL)
    rrm.cache.Wait()
}
```

Risk: Transient provider or network issues can cause a legitimate request (e.g. same eth_call params) to be banned for 6 hours. For enterprises, this is a long window where that request shape cannot benefit from retries or provider recovery. The ban is global to the process, not per-provider.

No Retries for Stateful and CrossValidation

Stateful and CrossValidation modes never retry. One timeout or bad response from a provider can force a user-visible failure.

Code (from `smartrouter_relay_state_machine.go`):

```
switch srsm.selection {
case relaycore.CrossValidation:
    return false  // No retries
case relaycore.Stateful:
    return false  // No retries
case relaycore.Stateless:
    // ... batch check, unsupported method, max retries ...
    return true
}
```

Risk: For stateful or cross-validated flows (e.g. consistency-critical reads), a single provider blip causes immediate failure. Enterprises often use these modes for correctness; the lack of any retry or circuit breaker amplifies the impact of flaky providers.

Batch Requests Never Retried

Batch JSON-RPC requests have retry disabled by default (`DisableBatchRequestRetry == true`). Batch requests are not hashed for caching, and retrying batches can be dangerous for stateful operations, so the codebase disables it.

Code (`protocol/relaycore/selection.go`):

```
var DisableBatchRequestRetry = true
// In state machine: if relaycore.DisableBatchRequestRetry &&
srsm.protocolMessage.IsBatch() { return false }
```

Risk: Exchanges and wallets often use batch RPC (e.g. multiple `eth_getBalance` in one call). A single timeout or provider error fails the entire batch with no retry or failover at the router level.

Health Check Cadence

Provider health is checked on a fixed interval; the default is 5 minutes (`RelayHealthIntervalFlag`). Until the next run, the router may keep sending traffic to a provider that has already failed.

Risk: In production, 5 minutes of traffic to a dead or degraded provider is unacceptable for custodians and exchanges. Detection and failover are delayed, and there is no “fast path” (e.g. failure-count based) to stop using a provider before the next health run.

No Per-Provider Circuit Breaker

The state machine retries by creating new relay states and trying other providers, but there is no circuit breaker: a provider that is failing repeatedly is not temporarily excluded from selection for a cooldown period. Traffic continues to be sent until the health check (or optimizer) eventually marks it bad.

Risk: Under partial outages or flapping, the same failing provider keeps receiving requests, increasing latency and error rate for users and delaying effective failover. Enterprises expect circuit breakers to isolate bad dependencies quickly.

Processing Timeouts and Context Propagation

Processing timeout is derived from chain block time and request type (`getProcessingTimeout`); it is then applied via `LowerContextTimeout` and context propagation through the relay pipeline. Under load, many in-flight requests can approach their timeout; when the context is cancelled, retries and “ban” logic fire in bursts.

Risk: Timeout stacking: under sustained load, a wave of timeouts can trigger many retries and bans (Stateless), or immediate failures (Stateful/CrossVal), and can make the system look worse than the underlying provider availability.

Failure Modes Under Production Stress

All Static + Backup Providers Degraded

If both static and backup providers are slow or failing, health checks (every 5 minutes) may not yet have marked them unhealthy. In Stateless mode, requests retry up to 10 times across providers, then get banned for 6 hours. In Stateful/CrossValidation, the first failure is user-visible. Recovery is only detected on the next health run, so there is no sub-5-minute “back to healthy” signal.

3.2 Thundering Herd / Stampede

After a provider recovers or after cache expiry, a burst of requests can hit the same provider. There is no rate limiting or backpressure at the Smart Router level, so the recovered provider (or the "best" one by QoS) can be overloaded, causing new timeouts and bans.

Timeout Stacking

Under high concurrency, many relays approach processingTimeout. When contexts expire, the state machine triggers retries (Stateless) or returns failure (Stateful/CrossVal). Retries consume more provider capacity and can add more banned hashes. The system can appear to degrade faster than the actual provider failure rate.

Batch and Stateful Sensitivity

Batch RPC and Stateful/CrossVal flows have no retry. One timeout or bad response fails the whole operation. For exchanges doing batch balance or tx checks, this makes the router a single point of failure for that request type.

RelayRetriesManager "Ban" Scope

The hash-based ban is process-wide and long-lived (6h). After a transient outage, identical requests (same method and params) remain banned even if all providers are healthy again. Operators cannot "reset" a request shape without restarting or changing the request.

Prioritized 90-Day Hardening Plan

Phase 1 (Days 1–30): Observability and Safety Nets

- **Metrics and SLOs:** Document and expose existing Prometheus metrics (lava_consumer_total_relays_serviced, lava_consumer_total_errored, latency, per-provider and per-chain). Add SLO-style metrics (e.g. error rate, p99 latency) and optional alerting rules so enterprises can run to a target (e.g. 99.9% success, p99 < 2s).
- **Per-provider circuit breaker:** Implement a circuit breaker (failure count or error-rate threshold + cooldown). When a provider exceeds the threshold, stop sending it traffic for a configurable period; after cooldown, allow one probe before resuming. This reduces impact of flaky or degraded providers.
- **Health check tuning:** Keep the health check interval configurable; consider a shorter default (e.g. 1m) for "enterprise" presets, with documentation that shorter intervals increase probe traffic.

Phase 2 (Days 31–60): Retry and Timeout Policy

- **Retry policy:** Cap retries per request (e.g. keep 10 but make it configurable), consider exponential backoff between attempts, and where feasible differentiate idempotent vs non-idempotent methods (e.g. no retry for eth_sendRawTransaction by default).
- **RelayRetriesManager:** Shorten TTL or narrow scope (e.g. per-provider or per-request-id) so transient failures do not ban a request shape for 6 hours. Optionally make TTL and scope configurable.
- **Batch retry:** Document the current "no batch retry" behavior and risks. Optionally add a flag to enable batch retry for operators who accept the statefulness risk (default off).

Phase 3 (Days 61–90): Resilience and Operations

- **Rate limiting / backpressure:** Introduce per-client or per-chain limits (or connection backpressure) at the router to avoid stampedes when a provider recovers or when cache expires.
- **Runbooks and deployment:** Provide runbooks for "all providers down", "high error rate", "high latency", and deployment guidance (config validation, recommended backup layout, metrics and alerts).
- **Testing:** Add minimal chaos or integration tests (e.g. kill one provider, verify failover and circuit breaker behavior) to guard against regressions.

What Not to Focus On Initially

- Do not rebuild the Smart Router in another language or replace relaycore in the first 90 days. Focus on configuration, retry/health, and observability within the existing Go codebase.
- Do not prioritize new API protocols or new chains. Hardening the existing JSON-RPC/REST path for enterprise use has a higher impact.
- Do not add heavy ML or dynamic pricing. Keep provider selection and QoS as-is except for circuit breaker and retry tuning.
- Do not spend 90 days on Kafka or analytics pipelines. Ensure Prometheus and health checks are solid and documented first; analytics can follow once reliability and observability are in place.